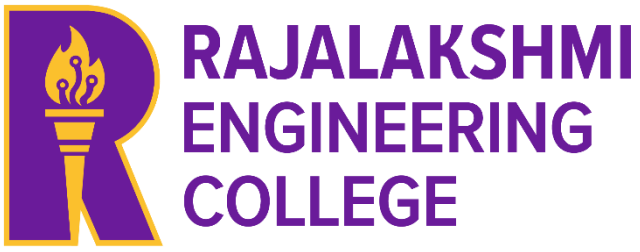
RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23331 - DESIGN AND ANALYSIS OF ALGORITHMS

Laboratory Record Note Book

Name :	:	PRAKASH A
Year / Branch / Section	:	II / BE Computer Science and Design / -
University Register No.	:	2116251701041
College Roll No. :	:	251701041
Semester:	:	III
Academic Year :	:	2026 - 2027

DIVIDE AND CONQUER

EXPERIMENT 11

1-Number of Zeros in a Given Array

Divide and Conquer

Started on	Wednesday, 30 September 2026, 9:13 PM
State	Finished
Completed on	Wednesday, 30 September 2026, 9:17 PM
Time taken	4 mins 13 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int countZeroes(int a[], int low, int high)
{
    if (low > high)
        return 0;

    int mid = (low + high) / 2;

    if (a[mid] == 0)
        return (high - mid + 1) + countZeroes(a, low, mid - 1);

    return countZeroes(a, mid + 1, high);
}

int main()
{
    int m;
    scanf("%d", &m);

    int a[m];

    for (int i = 0; i < m; i++)
        scanf("%d", &a[i]);

    printf("%d", countZeroes(a, 0, m - 1));

    return 0;
}
```

	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔
✔	8 0 0 0 0 0 0 0 0	8	8	✔
✔	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

EXPERIMENT 12

2-Majority Element

Divide and Conquer

Started on	Wednesday, 30 September 2026, 9:18 PM
State	Finished
Completed on	Wednesday, 30 September 2026, 9:19 PM
Time taken	47 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Given an array `nums` of size `n`, return *the majority element.*

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int majorityElement(int a[], int n)
{
    int count = 0, candidate = 0;

    for (int i = 0; i < n; i++)
    {
        if (count == 0)
            candidate = a[i];

        if (a[i] == candidate)
            count++;
        else
            count--;
    }

    return candidate;
}

int main()
{
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("%d", majorityElement(a, n));

    return 0;
}
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

EXPERIMENT 13

3-Finding Floor Value

Divide and Conquer

CS23331-Design and Analysis of Algorithms-2025 Batch-CSD

Started on	Wednesday, 30 September 2026, 9:20 PM
State	Finished
Completed on	Wednesday, 30 September 2026, 9:22 PM
Time taken	1 min 42 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int findFloor(int a[], int low, int high, int x)
{
    if (low > high)
        return -1;

    int mid = (low + high) / 2;

    if (a[mid] == x)
        return a[mid];

    if (a[mid] > x)
        return findFloor(a, low, mid - 1, x);

    int result = findFloor(a, mid + 1, high, x);

    if (result == -1)
        return a[mid];

    return result;
}

int main()
{
    int n, x;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    printf("%d", findFloor(a, 0, n - 1, x));

    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

EXPERIMENT 14

4-Two Elements sum to x

Divide and Conquer

CS23331-Design and Analysis of Algorithms-2025 Batch-CSD

Started on	Wednesday, 30 September 2026, 9:24 PM
State	Finished
Completed on	Wednesday, 30 September 2026, 9:25 PM
Time taken	47 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 Correct Mark 1.00 out of 1.00 Flag question

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int findPair(int a[], int low, int high, int x, int *e1, int *e2)
{
    if (low >= high)
        return 0;

    int sum = a[low] + a[high];

    if (sum == x)
    {
        *e1 = a[low];
        *e2 = a[high];
        return 1;
    }

    if (sum < x)
        return findPair(a, low + 1, high, x, e1, e2);

    return findPair(a, low, high - 1, x, e1, e2);
}

int main()
{
    int n, x, e1, e2;

    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    if (findPair(a, 0, n - 1, x, &e1, &e2))
    {
        printf("%d\n", e1);
        printf("%d\n", e2);
    }
    else
    {
        printf("No");
    }

    return 0;
}
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

EXPERIMENT 15

5-Implementation of Quick Sort

Divide and Conquer

CS23331-Design and Analysis of Algorithms-2025 Batch-CSD

Started on	Wednesday, 30 September 2026, 9:27 PM
State	Finished
Completed on	Wednesday, 30 September 2026, 9:27 PM
Time taken	44 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```c
#include <stdio.h>

void quickSort(int a[], int low, int high)
{
    if (low < high)
    {
        int i = low;
        int j = high;
        int pivot = a[low];

        while (i < j)
        {
            while (a[i] <= pivot && i < high)
                i++;

            while (a[j] > pivot)
                j--;

            if (i < j)
            {
                int temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }

        a[low] = a[j];
        a[j] = pivot;

        quickSort(a, low, j - 1);
        quickSort(a, j + 1, high);
    }
}

int main()
{
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quickSort(a, 0, n - 1);

    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.